```
#include <iostream>
#include <vector>
using namespace std;

int main() {
int m;
cout << "Enter number of data bits: ";
cin >> m;

vector<int> data(m);

cout << "Enter data bits: ";

for (int i = 0; i < m; i++)

   cin >> data[i];


int r = 0;

while ((1 << r) < m + r + 1)

   r++;


int n = m + r;

vector<int> code(n + 1, 0);


// Place data bits in non-parity positions

int j = 0;

for (int i = 1; i <= n; i++) {

   if ((i & (i - 1)) != 0)
```

```
        code[i] = data[j++];
}

// Calculate even parity bits
for (int i = 0; i < r; i++) {
    int p = 1 << i;
    int parity = 0;

    for (int k = 1; k <= n; k++) {
        if (k & p)
            parity ^= code[k];
    }

    code[p] = parity;
}

cout << "\nTransmitted Hamming Code: ";
for (int i = 1; i <= n; i++)
    cout << code[i];
cout << endl;

// Introduce a single-bit error
```

```
int errorPos;
cout << "Enter bit position to corrupt (1 to " << n << "): ";
cin >> errorPos;

if (errorPos < 1 || errorPos > n) {
    cout << "Invalid bit position!\n";
    return 0;
}

code[errorPos] ^= 1;

cout << "Received Corrupted Code: ";
for (int i = 1; i <= n; i++)
    cout << code[i];
cout << endl;

// Detect the error using syndrome
int syndrome = 0;

for (int i = 0; i < r; i++) {
    int p = 1 << i;
    int parity = 0;
```

```
    for (int k = 1; k <= n; k++) {
        if (k & p)
            parity ^= code[k];
    }

    if (parity)
        syndrome += p;
}

if (syndrome == 0) {
    cout << "No error detected.\n";
} else if (syndrome <= n) {
    cout << "Error detected at bit position: "
         << syndrome << endl;

    // Correct the corrupted bit
    code[syndrome] ^= 1;

    cout << "Corrected Hamming Code: ";
    for (int i = 1; i <= n; i++)
        cout << code[i];
```

```
    cout << endl;
} else {
    cout << "Error detected, but position is invalid.\n";
}

// Extract original data bits
cout << "Recovered Data Bits: ";
for (int i = 1; i <= n; i++) {
    if ((i & (i - 1)) != 0)
        cout << code[i];
}
cout << endl;

return 0;
}
```